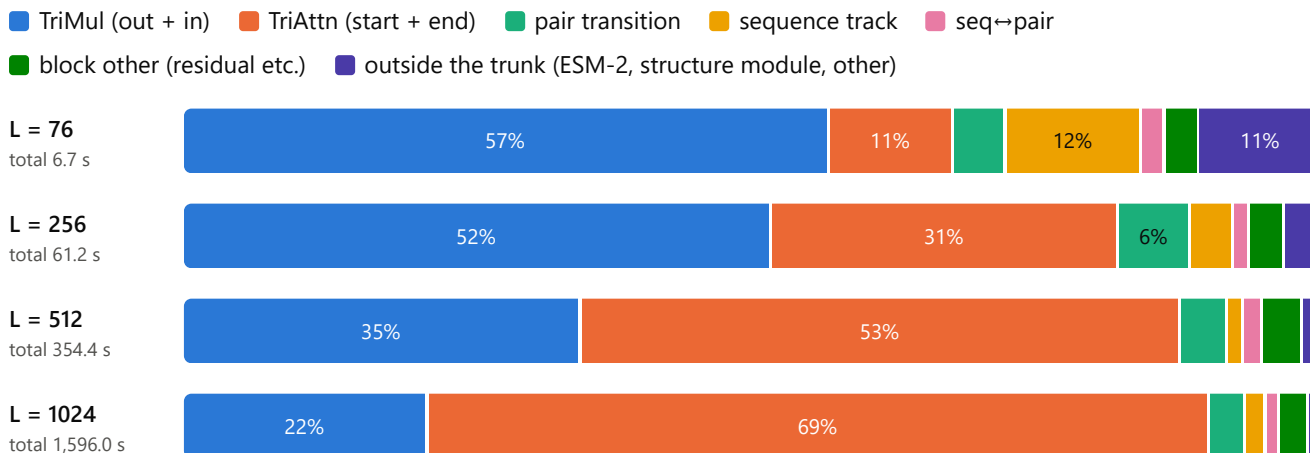


ESMFold Time Breakdown and Findings (workload characterization)

1. Time share of the whole pipeline vs. sequence length L



Each bar is the total time of one complete inference (amino-acid sequence in, 3D structure out), split into 7 non-overlapping groups. Lengths: 76 (ubiquitin), 256, 512, 1024. Per-category seconds are in Table 1.

1. The triangle operations (blue + orange) take a growing share as L increases, reaching 90% at L = 1024.
2. TriMul (blue) shrinks while TriAttn (orange) grows; they cross between L = 256 and 512.
3. ESM-2 and the structure module (purple) are barely visible for L ≥ 256.

Table 1 Time and share of every category (sum over 192 block calls)

Category	L = 76		L = 256		L = 512		L = 1024	
	s	%	s	%	s	%	s	%
Triangle Multiplication (outgoing)	1.92	28.7	16.36	26.7	62.12	17.5	171.89	10.8
Triangle Multiplication (incoming)	1.89	28.2	15.31	25.0	61.95	17.5	172.11	10.8
Triangle Attention (starting node)	0.37	5.5	9.38	15.3	93.14	26.3	487.88	30.6
Triangle Attention (ending node)	0.36	5.4	9.33	15.2	93.82	26.5	609.17	38.2
pair transition (mlp_pair)	0.31	4.6	3.87	6.3	14.72	4.2	51.05	3.2
sequence attention	0.40	6.0	1.53	2.5	3.75	1.1	18.69	1.2
sequence transition (mlp_seq)	0.39	5.8	0.78	1.3	1.22	0.3	10.12	0.6
LayerNorm (sequence)	0.01	0.1	0.02	0.0	0.04	0.0	0.05	0.0
sequence → pair	0.11	1.6	0.64	1.0	4.20	1.2	11.85	0.7
pair → sequence	0.03	0.4	0.22	0.4	1.73	0.5	6.32	0.4
block other (residual additions etc.)	0.20	3.0	1.88	3.1	12.43	3.5	41.07	2.6
ESM-2 (language model)	0.52	7.8	1.18	1.9	2.88	0.8	6.40	0.4
structure module	0.19	2.8	0.61	1.0	2.06	0.6	8.03	0.5
other (embedding, recycling etc.)	0.02	0.3	0.07	0.1	0.39	0.1	1.36	0.1
Total time	6.7	100	61.2	100	354.4	100	1,596.0	100
Peak memory (whole process)	19.9 GB		22.2 GB		27.4 GB		60.7 GB	

2. Findings

Finding 1 The bottleneck of the pipeline is the triangle operations on the pair representation

- For $L \geq 256$, TriMul + TriAttn take **82–90%** of the whole pipeline; the 48 blocks together take 97–99%.
- ESM-2 + structure module: under 3% for $L \geq 256$. ESM-2 holds 80% of all parameters but under 1% of the time: parameter count sets how much memory the weights need, while time is set by the intermediate data (pair representation L^2 , triangle operations L^3).
- Consistent with LightNobel accelerating only the trunk, not ESM-2 or the structure module.

Finding 2 The bottleneck shifts with length: TriMul dominates short sequences, TriAttn dominates long ones

- TriMul share 57% $\rightarrow$ 52% $\rightarrow$ 35% $\rightarrow$ 22%; TriAttn 11% $\rightarrow$ 31% $\rightarrow$ 53% $\rightarrow$ 69% ($L = 76 \rightarrow 1024$).
- The two cross between $L = 256$ and 512.
- Reason: both perform L^3 work, but TriMul's data is only L^2 (each element is reused L times), whereas TriAttn produces L^3 -sized logits (17 GB at $L = 1024$), so its data movement grows as L^3 .

Finding 3 TriAttn: memory-bandwidth bound, already close to the ceiling

Observations

- tri_att_start ($L = 1024$): add_ 31% + softmax 28% = 59% spent reading and writing logits; matrix multiplication takes only 27%.
- add_ reaches a nominal bandwidth of about 112 GB/s (68.7 GB counted $\div$ 0.616 s measured), against a machine ceiling of about 100 GB/s (same operation, same memory placement) $\rightarrow$ bandwidth is saturated. Roofline efficiency 58–88%.
- Peak memory at $L = 1024$ is 60.7 GB, mostly the unchunked logits (17 GB per copy).
- At $L = 1024$, tri_att_end is 583 ms slower than start; 578 ms of it (99%) comes from add_ (1,194 vs 616 ms). At $L = 512$ the two are equal.

Cause

- The softmax dependency needs only one row (4 KB at $L = 1024$); the 17 GB of traffic comes from executing each operation over the entire tensor before starting the next.

Finding 4 TriMul: most of its time is lost to data layout, not computation; fixing the layout alone saves about 10–13% of total time (L = 1024) and 19–24% (L = 512)

Observations

- `copy_` (copies after permute) takes **50–58%** of TriMul; the `einsum` itself only 14–17%.
- Roofline efficiency is only 16–24%, the lowest of the three operations.
- A permuted copy runs at only 9 GB/s versus 76–82 GB/s for a contiguous copy, about 9× slower.
- **Control experiment:** a matrix multiplication of the same shape ($128 \times 1024 \times 1024 \times 1024$) with data already in the right layout takes only **63 ms** (the machine ceiling); the HF implementation takes `einsum` 149 ms + `copy` 436 ms = **585 ms**, a **9.3×** gap. The shape is not the problem; the entire loss is in the layout.

Cause

- `bmm` requires `[c][i][k]`, but the data is stored as `[i][k][c]`. `a` and `b` are each transposed once and the result is transposed back once: 3 copies in total.
- Adjacent elements of the same channel are 512 bytes apart, so each 64-byte cache line fetched yields only 4 useful bytes.

Estimated benefit (fixing the layout only)

- One `tri_mul_out` call (L = 1024, 881 ms) = group A (`einsum` + copies) 585 ms + group B (linear, gating, LayerNorm) 296 ms.
- Conservative case (remove the 2 copies before the `einsum`, still transpose the result back once): group A = 63 + 124 = 187 ms → TriMul 1.82× faster → total time reduced by **9.7%**.
- Ideal case (all 3 copies removed): group A = 63 ms → TriMul 2.45× faster → total time reduced by **12.8%**.
- L = 512: the ideal `einsum` was not measured; estimated from an efficiency range, total time reduced by 19–24%.
- Group B is assumed unchanged; fusing group B as well (no intermediate write-back) would save more, so these are conservative estimates. Upper bound for optimizing TriMul alone: at most 1.54× at L = 512 and 1.28× at L = 1024.